

Traffic Simulator - Part 2

Carried out within the scope of the "Telecommunications Networks" course.

André Silva **Leonor Guedes** **Rafael Parody**
up202107419 up202107691 up202502656

1 Introduction

This work aims to simulate a packet oriented system under three conditions: a loss system based on Erlang-B distribution, a waiting system with an infinite queue based on Erlang-C distribution and finally a waiting system with a finite queue. It was considered a system with an arrival rate $\lambda = 200$ packets/ms an average transmission duration $d_m = 0.008$ ms.

Since the arrival rate is very high, a large number of samples was required to obtain statistically stable results and ensure convergence of the estimated metrics. Therefore, all simulations were performed with 100000 samples, which provided a good compromise between accuracy and computational efficiency.

The source code developed for this lab work is available at: https://github.com/mLeOn0r/Traffic_Simulator.

2 Loss System

The Erlang-B simulation models a loss system through a discrete-event approach, where each event represent either a call arrival or departure. The generation of events was implemented using the code developed for the previous part of the Lab work, which was adapted to generate this two types of events and gathered in the function `generate_events()`.

At the beginning of the simulation, the first event generated is always an arrival, then the algorithm is iteratively processed until the desired number of samples is reached. When a call arrives, the total number of arrivals is incremented. If there is any service available, meaning the number of busy channels is smaller than the total number of channels N , the call is accepted and a corresponding departure event is generated. In the case that all channels are busy, the call is blocked and the number of blocked calls incremented. In order to maintain the arrival process, in both situations a new arrival event is scheduled.

When the next event in the list is a departure, one of the channels then becomes available, which is represented by decrementing the number of busy channels, allowing the next arrival to be accepted.

After the simulation, the blocking probability is calculated as the ratio between the number of blocked calls and the total of arrivals, given in percentage. Table 1 presents the results obtained and the expected values for different number of channels.

Table 1: Blocking probability obtained vs expected.

N	Simulated	Theoretical
1	61.98 %	61.54 %
2	32.78 %	32.99 %
3	15.73 %	14.96 %
4	5.01 %	5.65 %
5	1.56 %	1.77 %
6	0.40 %	0.47 %
7	0.06 %	0.11 %
8	0.03 %	0.02 %

3 Waiting System with an infinite length queue

The Erlang-C simulation models a system with unlimited waiting queue with first-in-first-out service. Events were generated as described before and the simulation begins by generating the first arrival.

When an arrival occurs, the total number of arrivals is incremented and the availability of the channels is checked. If there is any channel available, the call is accepted immediately, the number of busy channels incremented and a new departure is generated. If all channels are busy, the call is delayed, added to a waiting queue and the delayed counter is incremented. In both cases, a new arrival event is generated.

When a departure event occurs and the waiting queue is empty, the number of busy channels is decremented. Otherwise, the event is removed from the queue and a new departure event is generated. The individual waiting time is obtained by subtracting the arrival time stored in the queue from the time the new departure is generated, as it represents the moment the event left the queue.

The following table presents the performance metrics obtained from the Erlang-C simulation for different numbers of servers. With only one channel the system is unstable, since the arrival rate exceeds the service capacity, leading to much larger delay values. As the number of channels increases, both the delay probability and average delay decrease quickly, demonstrating the expected improvement in system performance. The threshold value A_x was chosen to match d_m , allowing the probability $P_r(A > A_x)$ to represent the fraction of calls whose waiting time exceeds the average service duration.

Table 2: Performance metrics obtained from the Erlang-C simulation.

N	Delay Probability		Average Delay	Probability A > 0.008 ms
	Simulated	Theoretical	Simulated	
1	100.00 %	100.00 %	58.30 ms	62.57 %
2	71.50 %	71.11 %	14.43 us	48.55 %
3	27.53 %	27.38 %	1.563 us	6.82 %
4	8.88 %	9.07 %	0.292 us	0.78 %
5	2.38 %	2.59 %	0.0546 us	0.07 %
6	0.58 %	0.64 %	0.01025 us	0.00 %
7	0.11 %	0.14 %	0.001370 us	0.00 %
8	0.03 %	0.03 %	0.000358 us	0.00 %
9	0.00 %	0.00 %	0.000028 us	0.00 %

To visualize the distribution of delays, histograms were created. The average delay (A_m) was estimated for each number of channels N following a theoretical approach, so the maximum value represented and the bin width of each histogram could be adjusted accordingly. This adjustment made the results easier to visualize, preventing all values from accumulating in the last interval.

After analysing the delay values obtained for a system with N = 1 (Figure 1, Appendix A.1), it was confirmed the unstable behaviour expected. Comparatively, the distribution of delays for systems with larger number of channels (Figure 2, Appendix A.1), showed that the system operates under stable conditions, so minimal waiting times were observed.

4 Waiting System with a finite length queue

To better reflect real world systems, the last simulation consisted in a general case, extended from the previous Erlang-C simulation by introducing a waiting queue with a limited capacity, L. Event generation and processing follow the same logic explain in previous sections but with a small difference: if all channels are busy and the length of the waiting queue is less than L the call is admitted, otherwise the call is blocked.

The same performance metrics were obtained along with the blocking probability and compared with the previous results (Figure 3 and Table 4, Appendix A.2). It was possible to confirm that the results obtained from the simulation behaved consistently with the theoretical values. The Erlang-B model can be seen as a particular case of the general case when $L = 0$, where all calls arriving to a busy system are blocked. Similarly, Erlang-C corresponds to the case where $L \rightarrow \infty$, allowing an unlimited waiting queue in which no calls are rejected.

The general case was also used to determine the minimum queue length required to achieve a blocking probability of approximately 1%. For each number of channels N, the queue length was progressively incremented until the blocking probability dropped below this threshold. With this analysis it was identified the waiting queue size necessary to maintain the quality of service, while balancing system performance and resource usage. Table 3 shows the results obtained.

Table 3: Queue length requirements for 1% Blocking Probability

N	Length	Blocking Probability
2	12	0.98%
3	5	0.57%
4	2	0.89%
5	1	0.56%

Appendix

A.1 Erlang-C - Histograms of delays

This subsection aims to present two of the histograms generated in the Erlang-C simulation. Additional histograms and comparison plots can be found in the project's GitHub repository at *Traffic_Simulator/lab1/Part2/plots/img/Erlang C*.

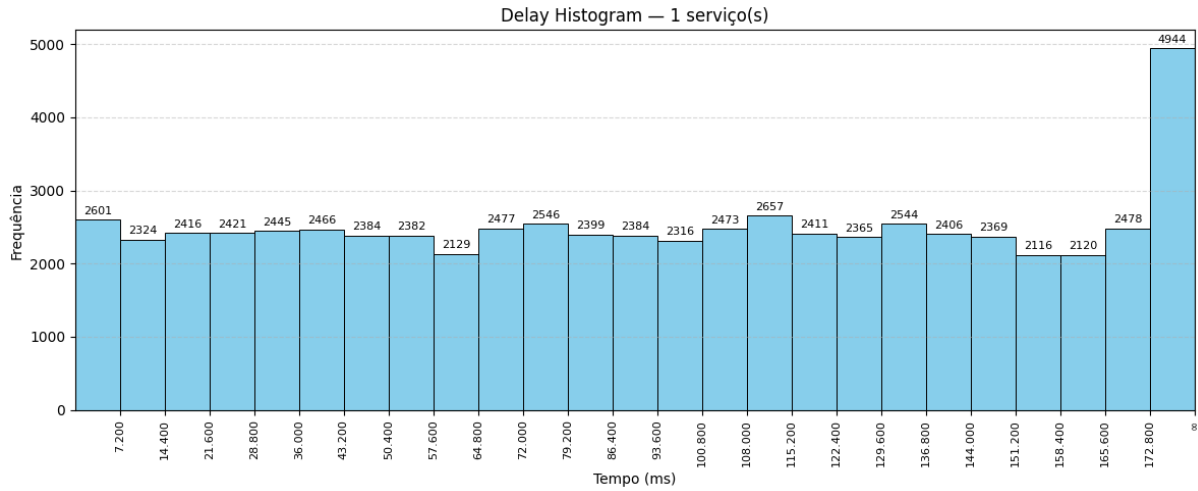


Figure 1: Delay histogram for $N = 1$.

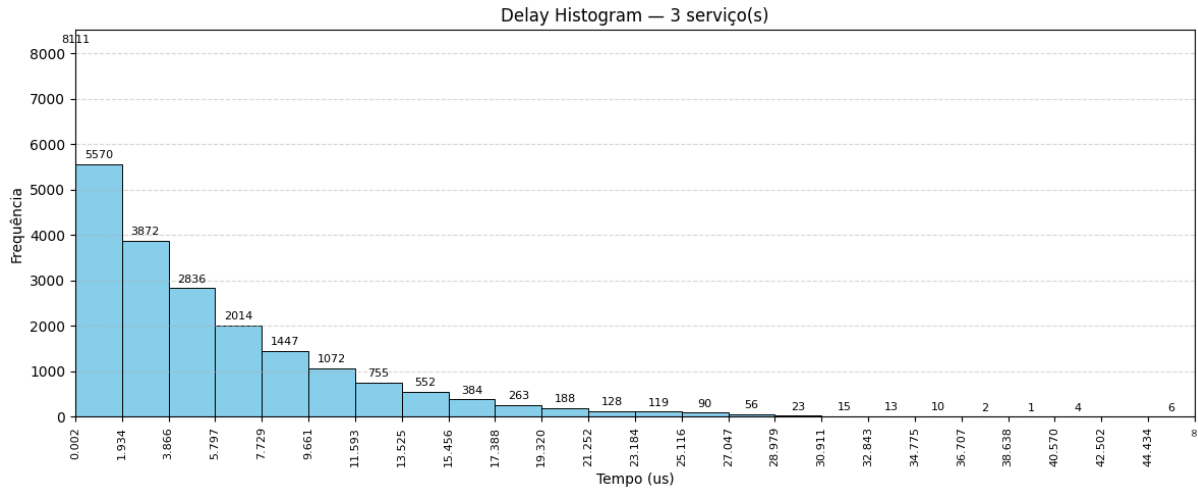


Figure 2: Delay histogram for $N = 3$.

A.2 General Case results

This subsection aims to present a comparison of the results obtain in the Erlang-B and Erlang-C simulations and the corresponding special case results of the general model. Additional histograms values and comparison plots can be found in the project's GitHub repository at *Traffic_Simulator/lab1/Part2/Results* and *Traffic_Simulator/lab1/Part2/plots/img/General Case*, respectively.

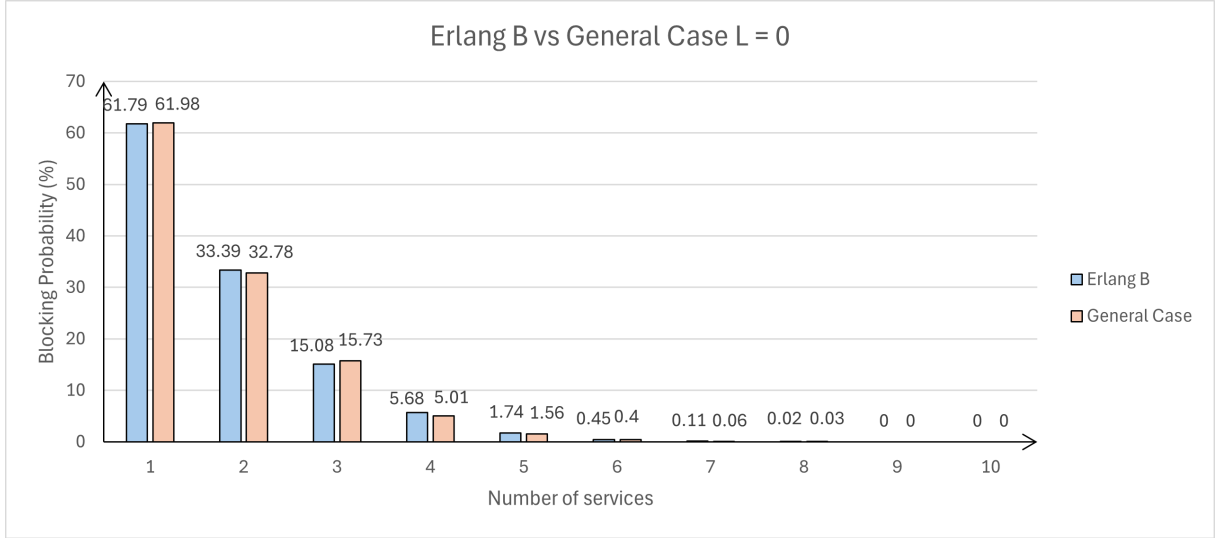


Figure 3: Erlang-B simulation vs General case with L=0.

Table 4: Erlang-C simulation vs General Case with L = 100000.

N	Delay Probability		Average Delay		Probability A >0.008 ms	
	Erlang-C	General Case	Erlang-C	General Case	Erlang-C	General Case
1	100.00 %	100.00 %	58.30 ms	58.08 ms	62.57 %	62.53 %
2	71.50 %	70.51 %	14.43 us	14.90 us	48.55 %	47.38 %
3	27.53 %	26.68 %	1.563 us	1.486 us	6.82 %	6.39 %
4	8.88 %	9.04 %	0.292 us	0.305 us	0.78 %	0.87 %
5	2.38 %	2.43 %	0.0546 us	0.0544 us	0.07 %	0.06 %
6	0.58 %	0.53 %	0.0102 us	0.00810 us	0.00 %	0.00 %
7	0.11 %	0.16 %	0.00137 us	0.00282 us	0.00 %	0.00 %
8	0.03 %	0.04 %	0.000358 us	0.000399 us	0.00 %	0.00 %
9	0.00 %	0.01 %	0.000028 us	0.000082 us	0.00 %	0.00 %